

High Performance Programming

Lecture 4:
Analytical Modelling of Parallel Systems and Applications



AALBORG
UNIVERSITET

Teaching plan

Parallel concepts, models and algorithms

- **Introduction to parallel processing (RA) - 13/02/2025**
- **Parallel processing: Basic data structures and communication (RA) – 20/02/2025**
- **Paradigms for parallel programs (RA) – 27/02/2025**
- Analytical modelling of parallel systems and applications (RA) – 06/03/2025

High performance programming, hardware architecture verification

- Practical aspects of parallel programming (SNE) – 13/03/2025
- Distributed memory parallel programming (RA) – 20/03/2025
- Shared memory parallel programming (SNE) – 27/03/2025
- GPU programming & Vectorization (SNE) – 03/04/2025
- Testing and verification (RA) – 24/04/2025

Workshop: Design and implementation of high-performance programs – 01/05/2025

- Problems released – 27/03/2025
- Report submission – 08/05/2025

Content

1. Analytical Modelling of Parallel Systems
2. Parallel Algorithms and Applications
3. Exercise

Analytical Modeling of Algorithms

- A sequential algorithm is evaluated by its runtime in function of its input size. $O(n)$, $\Omega(n)$, $\Theta(n)$
- The asymptotic runtime is independent of the computing architecture/platform.
- A parallel algorithm is evaluated by its runtime as functions of
 - the input size,
 - the number of processors,
 - the communication parameters.
- Overheads in parallel programs: **wasted computation, communication, idling and contention**
 - Load imbalance
 - Dependencies
 - Inter-process interactions

Sources of Overheads in Parallel Programs

Interprocess interactions: Processors working on any non-trivial parallel problem will need to talk to each other.

Idling: Processes may idle because of load imbalance, synchronization, or serial components.

Excess Computation:

- This is computation not performed by the serial version.
- This might be because the serial algorithm is difficult to parallelize, or that some computations are repeated across processors to minimize communication.

Performance Metrics for Parallel Algorithms: Recap

- Execution time → time elapsed between
 - beginning and end of execution on a sequential computer.
 - beginning of first processor and end of the last processor on a parallel computer

Total parallel overhead

$$T_o = pT_P - T_S$$

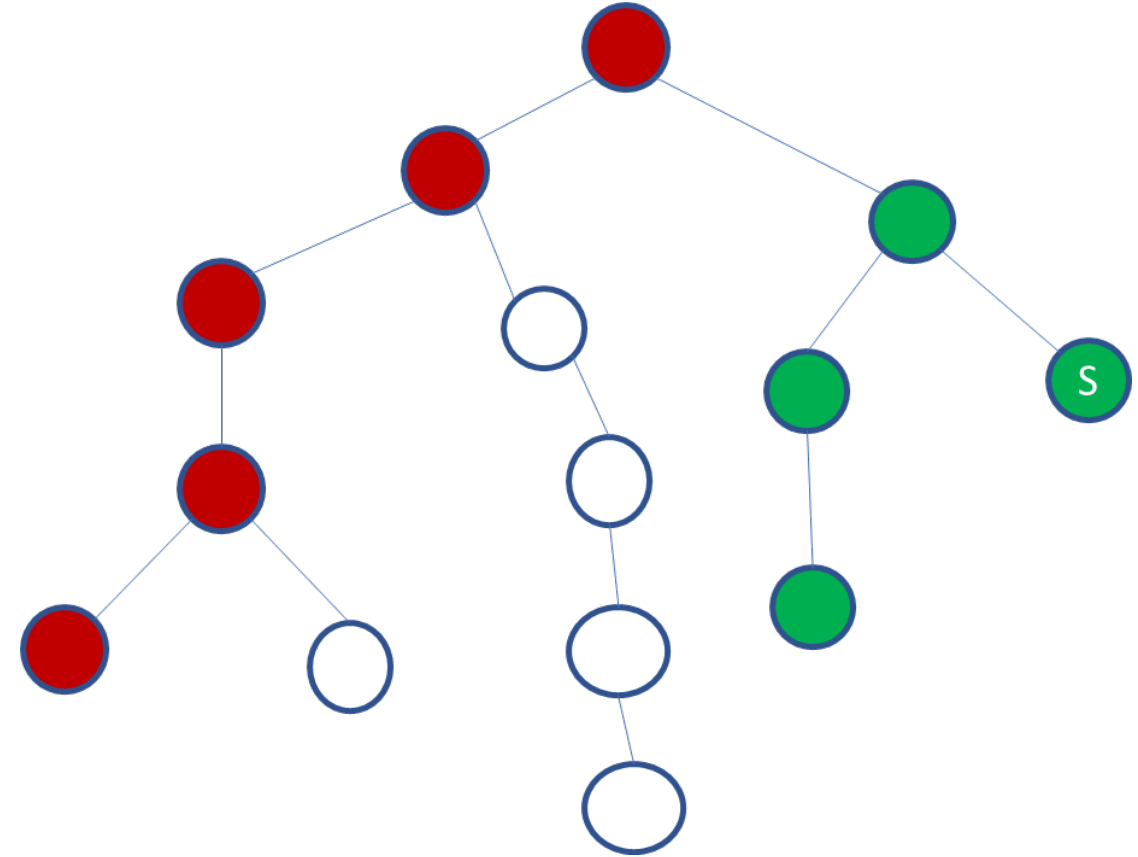
Total time spent by all processors or processing units

useful computation time

- A measure of benefit of parallelism → Speedup: $S = T_S/T_P$
- Example: Compute the sum of n elements
 - Best sequential algorithm: $O(n)$
 - Parallel algorithm: $O(\log_2(n))$
 - Speedup: $O(n/\log_2(n))$

Parallel Performance: Speedup

- In the book, speedup can never exceed p .
- Best parallel time: $T_P = T_S/p$
- In practice, super-linear speedup is possible. But how?
 - Serial algorithm does more work?
 - Effects from caches.
 - Decomposition method?
- Example: Depth-first Search (DFS)
 - 1 processing unit: $T_S = 14t_c$
 - 2 processing units: $T_2 = 5t_c$
 - A speed of $2.8 > 2$

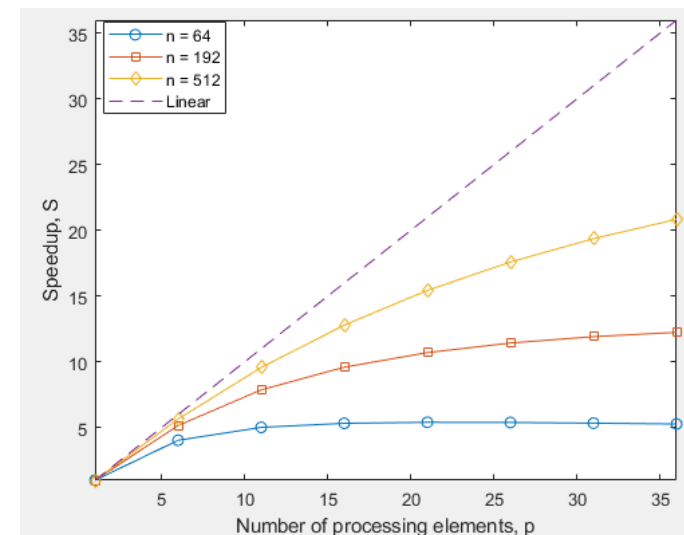


Parallel Performance: Efficiency

- Efficiency: $E = S/p$
 - Measures useful work time:
 - For the sum example: $E = O(1/\log_2(n))$
- Cost: $C = pT_P$
 - Also referred to as **work-time** or **processor-time** product
- Thus: $E = T_S/C$
 - A parallel algorithm is cost optimal if E is a constant

Scalability of Parallel Systems

- Scalability: measure of a parallel system's capacity to increase speedup in proportion to the number of processors.
- In practice:
 - Develop algorithms and test on small systems with small problems.
 - **What happens for the real large problems on large systems?**
 - Extrapolation of results is difficult
- Scaling characteristics of parallel programs
 - Efficiency rewritten $E = \frac{1}{1 + \frac{T_o}{T_s}}$
- For the sum of n elements problem: $E = \frac{1}{1 + \frac{2p \log_2(p)}{n}}$
- Efficiency can be kept constant when
 - the number of processors increases and the problem size increases.



Amdahl's Law (1967)

- The speedup of a program using multiple processors in parallel computing is *limited by the time needed for the serial fraction of the problem*.
- If a problem of size W has a serial component W_S , the speedup of the program is

$$T_P = \frac{W - W_S}{p} + W_S$$
$$S = \frac{W}{\frac{W - W_S}{p} + W_S} \leq \frac{W}{W_S}; \text{ as } p \rightarrow \infty$$

- Example: Assume $W_S = 20\%$, then $S \leq \frac{1}{0.2} = 5$; as $p \rightarrow \infty$

Implication:

- parallel computing is only useful when the number of processors is small, or when the problem is perfectly parallel, i.e., embarrassingly parallel

Gustafson's Law (1988)

- Also known as the Gustafson-Barsis's Law
- Any sufficiently large problem can be efficiently parallelized with a speedup

$$S = p - \alpha(p - 1)$$

- α is the serial portion of the problem
- Gustafson proposed a fixed time concept which leads to scaled speedup for larger problem sizes.
 - we use larger systems with more processors to solve larger problems

Scalable Parallel System and Cost Optimality

- Increase number of processors → decrease efficiency
- Increase problem size → increase efficiency
- **Can a parallel system keep efficiency by increasing the number of processors and the problem size simultaneously?**
 - Yes → scalable parallel system
 - No → non-scalable parallel system

A scalable parallel system:

- can maintain its efficiency constant when increasing the number of processors and the size of the problem.
- can always be made cost-optimal by adjusting the number of processors and the problem size.

Cost optimality:

- Cost optimal parallel systems have efficiency $\Theta(1)$.
- Algorithm for the n -element summation becomes cost optimal when $n = \Omega(p \log(p))$.

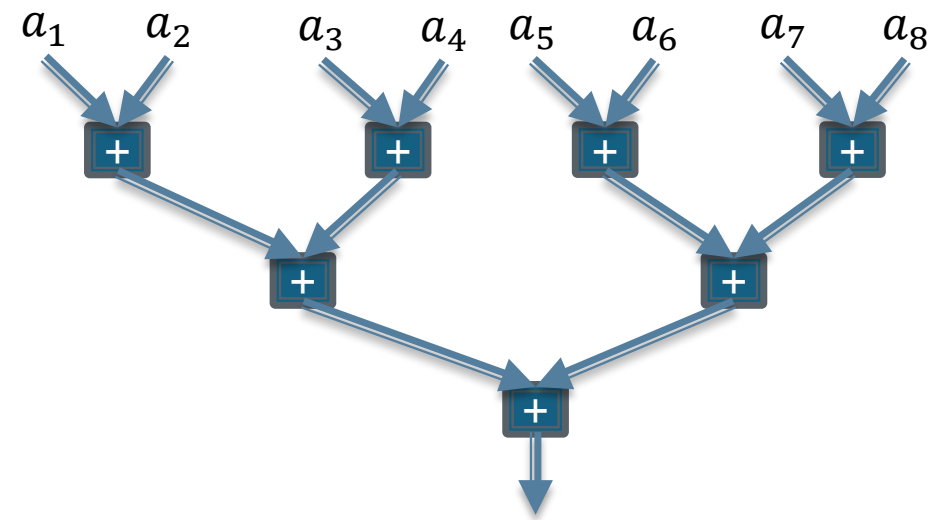
Parallel Algorithms and Applications

Reduce

- **Preconditions:** a binary associative operator $\oplus$ with identity I , a list of n -elements
- **Postcondition:** scalar s such that $s = a_1 \oplus a_2 \oplus a_3 \oplus \cdots \oplus a_n$
- **Example :** (operator $\oplus$ is $+$) and $A = [5, 4, 9, 0, 6, 1]$
- Reduce returns: $s = 25$
- Sequential reduction has complexity $O(n)$
- But the operator is associate!

Parallel Reduce

- How faster can we perform the reduce operation?
- Work: $W \in O(n)$
- Depth: $D \in O(\log_2(n))$
- Speed-up:
- Can we achieve higher speed-up with an infinite number of processing units?



Work-depth model/task dependency graph

Scan/Prefix Scan

- A good example of a computation that seems inherently sequential, but for which there is an efficient parallel algorithm

Preconditions: a binary associative operator $\oplus$ with identity I ,
an ordered list of n -elements $A = \{a_1, a_2, \dots, a_n\}$

Postcondition: n elements:

$s = \{I, a_1, (a_1 \oplus a_2), (a_1 \oplus a_2 \oplus a_3), \dots, (a_1 \oplus \dots \oplus a_{n-1})\}$ (exclusive) or

$s = \{a_1, (a_1 \oplus a_2), (a_1 \oplus a_2 \oplus a_3), \dots, (a_1 \oplus \dots \oplus a_n)\}$ (inclusive)

Example : (operator $\oplus$ is $+$) and $A = [15, 24, 9, 40, 6, -9]$

Scan returns: $S = [0, 15, 39, 48, 88, 94, 85]$ or $S = [\dots \dots \dots \dots \dots \dots \dots \dots \dots \dots]$

Prefix Scan: Application Example

- Suppose that we have a 100-inch sandwich to feed 10 people such that each person gets exactly what he/she wants: [5 7 4 9 30 6 5 2 10 3].
 - How do we cut the sandwich?
 - How much will be left?
- Solutions:
 1. Cut the sections sequentially: 5 inches first, 7 inches next
 2. Calculate prefix scan and cut in parallel:

Typical Applications of Scan

- Scan is a simple and useful parallel building block

✓

- Useful for many parallel algorithms:
 - Radix sort; quick sort
 - polynomial evaluation
 - solution of recurrences
 - Memory allocation to parallel threads
 - Allocation memory buffer for communication channels

Sequential Scan and Work Efficiency

- Trivial implementation
 - Loop over all elements of the input array and add the value of the previous element of the input to the sum from the previous element
- What is the work of the sequential scan?
 - Poorly suited for parallelization
- Parallel version of prefix-sum
 - Needs to be work-efficient → does no more work asymptotically than sequential equivalent

A Naïve Parallel Prefix-Sum

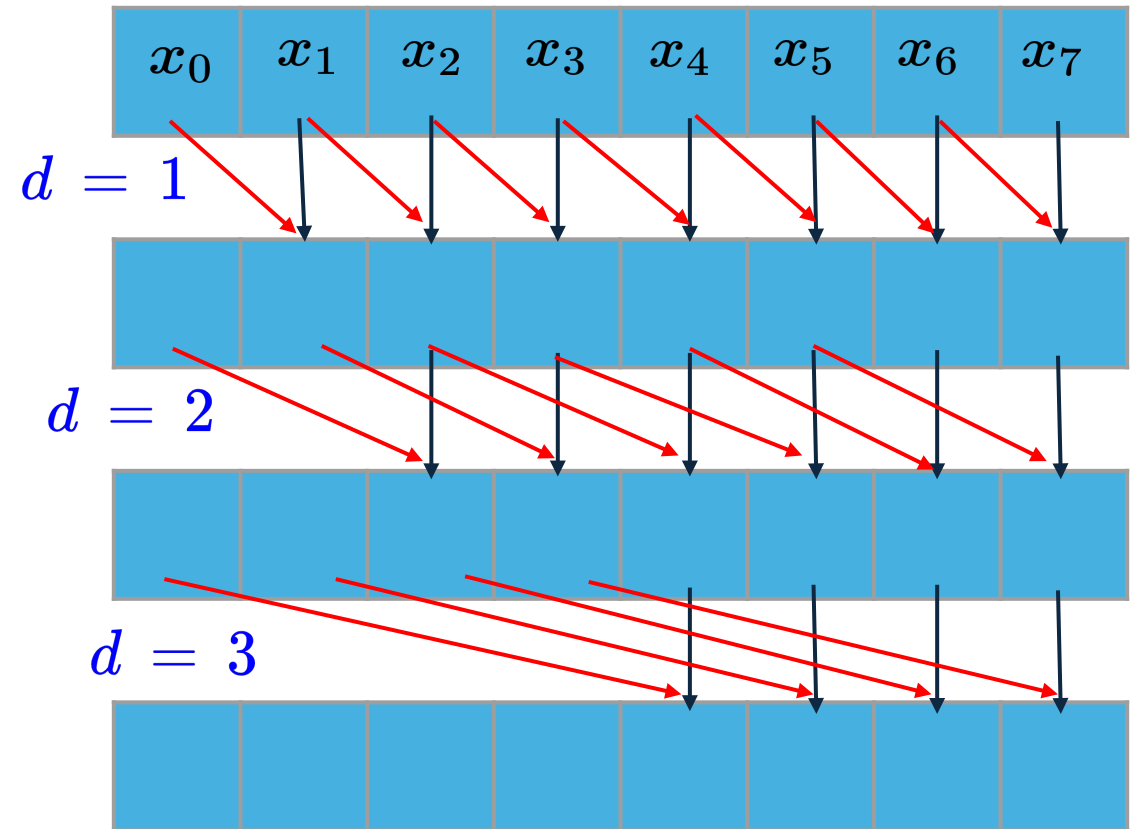
- Consider the naïve parallelization of prefix-sum

```
1: for  $d = 1 : \log_2(n)$  do
2:   for  $k = 1 : n$  do in parallel
3:     if  $k \geq 2^d$  then
4:        $x[k] = x[k - 2^{d-1}] + x[k]$ 
5:   End Parallel
```

- What is the work? $W(n) =$

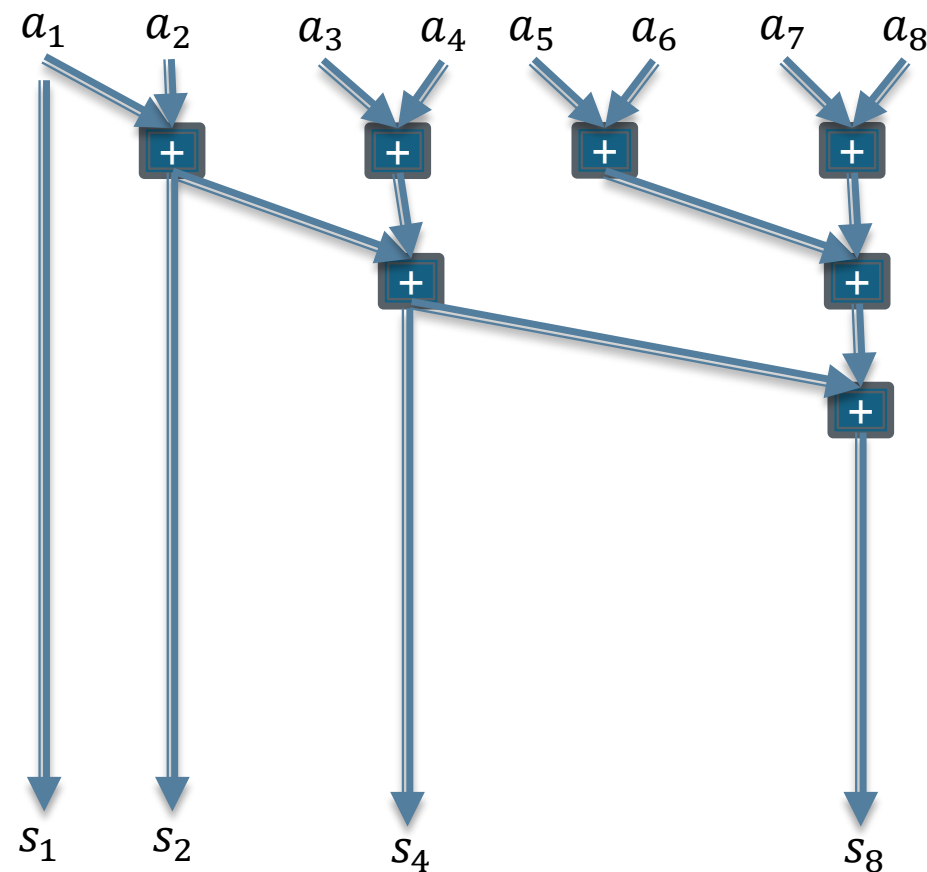
- Is the algorithm work efficient?

- Note:** Naïve parallel scan assumes that p is at least equal to n



A Work-Efficient Parallel Prefix Sum

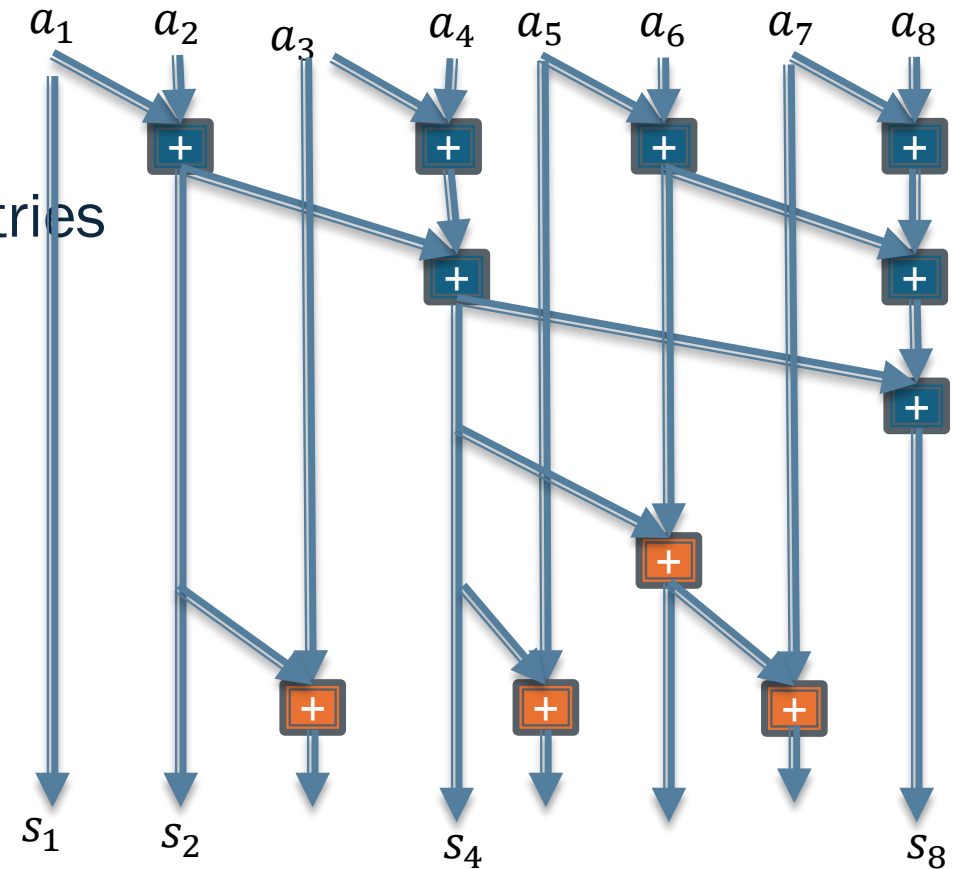
- Naïve parallel prefix sum has work complexity $O(n \log_2(n))$
 - A factor of $\log_2(n)$ larger compared to sequential prefix sum → **work inefficiency**
- A balanced binary tree allows us to design a work-efficient algorithm
 - Build a balanced binary tree on the input
 - Sweep it to and from the root to compute prefix sum
- Two phase algorithm:
 - parallel reduction (up-sweep)
 - Down-sweep



The prefix sum is incomplete – what do you notice?

A Work-Efficient Parallel Prefix-Sum

- How do we fill the gaps in the prefix sum?
- Create a similar tree to compute the remaining entries
- Time complexity
 - 2 tree structures, each with
 - $O(n)$ work
 - $O(\log_2(n))$



A Work-Efficient Parallel Prefix Sum

Stage 1: Up-sweep

```
1: For  $d = 0 : \log_2(n) - 1$  do  
2:   For  $k = 0 : 2^{d+1} : n - 1$  do in Parallel  
3:      $x[k + 2^{d+1} - 1] = x[k + 2^d - 1] + x[k + 2^d]$   
4:   End Parallel  
5: EndFor
```

Stage 2: Down-sweep

```
1:  $x[n - 1] \leftarrow 0$   
2: For  $d = \log_2(n) : 0$  do  
3:   For  $k = 0 : 2^{d+1} : n - 1$  do in Parallel  
4:      $t = x[k + 2^d - 1]$   
5:      $x[k + 2^d - 1] = x[k + 2^d]$   
6:      $x[k + 2^d] = t + x[k + 2^d]$   
7:   End Parallel  
8: End For
```

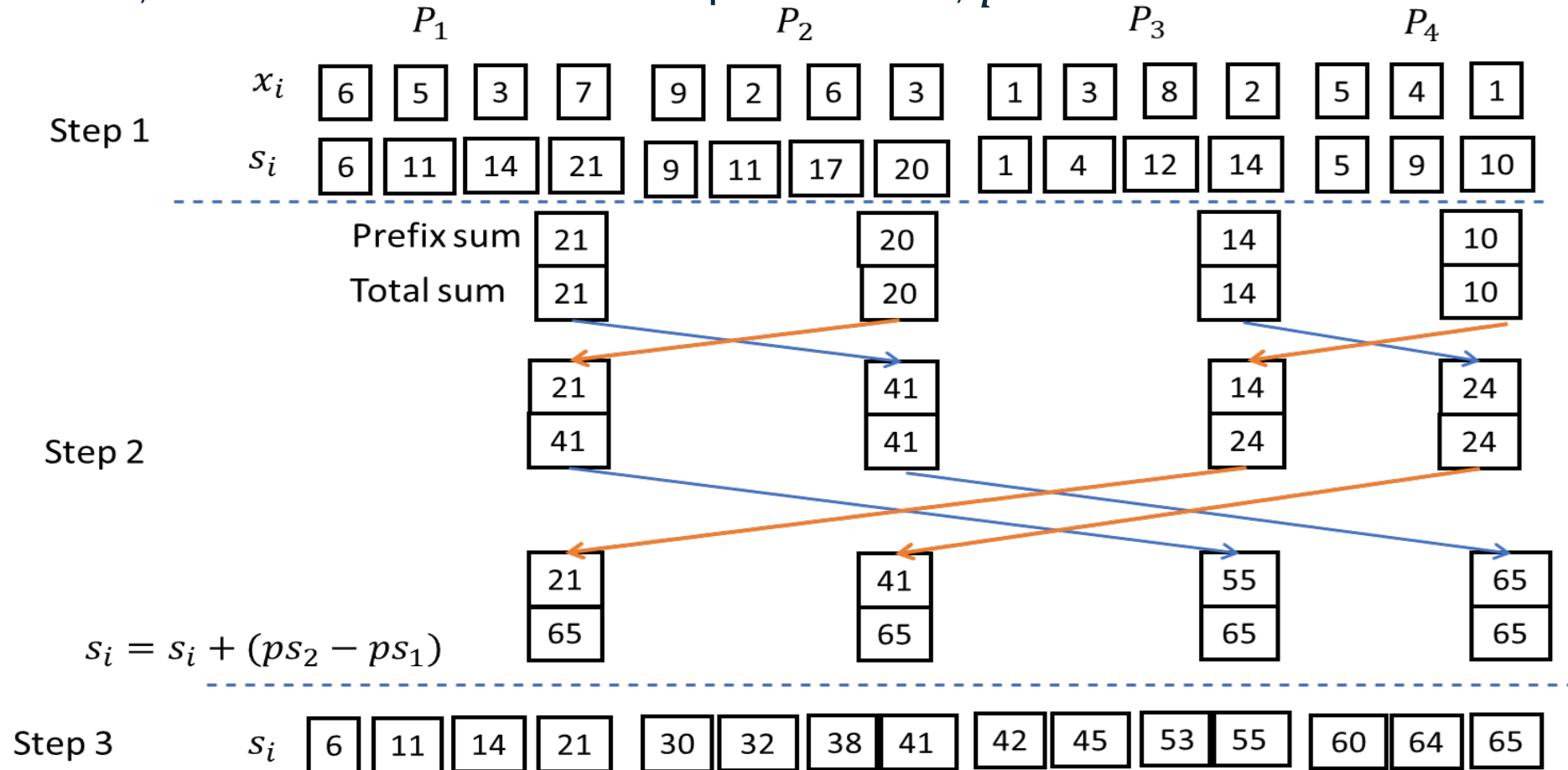
What is the work of each stage and of the entire algorithm?

Parallel Prefix Sum Algorithm: general solution

- Work-efficient parallel prefix
 - requires that array size is a power of two
 - Array size is limited by the number of processors
- Realistic case:
 - $n \gg p$
 - n is not a multiple of p
 - Each processor has either $\lceil n/p \rceil$ or $\lfloor n/p \rfloor$ elements.
 - p is not a power of 2.
 - In any communication phase, do nothing if the computed id of the processor to communicate with is $\geq p$.
- Steps (for simplicity think that each processor has n/p elements):
 1. Each processor computes the prefix sums of the n/p elements it has locally
 2. Using the last prefix sum on each processor, run a p -element parallel prefix algorithm
 3. On each processor, combine the result from the parallel prefix algorithm with each local prefix sum computed in Step 1.

Parallel Prefix Sum Algorithm: Example

- Problem size, $n = 15$ and number of processors, $p = 4$



Polynomial Evaluation

- Evaluation of a polynomial

- Preconditions:

- A real number x
 - n integer coefficients

$$\{a_0, \dots, a_{n-1}\}$$

- **Parallel Prefix for Polynomial Evaluation**

- Let the integer coefficients be evenly distributed on p processors
 - On processor p_i , we have $a_{in/p}$ to $a_{(i+1)n/p-1}$
 - Local computation required at each processor

$$s_i = \sum_{k=0}^{\frac{n}{p}-1} a_{\frac{in}{p}+k} x^{\frac{in}{p}+k}$$

- We use **parallel prefix** to get the required powers of x . But how?

Parallel Prefix for Linear Recurrences

- Preconditions: (1) Real numbers x_0, x_1
(2) Integer coefficients a, b
- Postcondition: Sequence $\{x_2, x_3, \dots, x_n\}$ such that $x_i = ax_{i-1} + bx_{i-2}$
- Solution:
 - We can re-write the recurrence relation as

$$\begin{bmatrix} x_i & x_{i-1} \end{bmatrix} = \begin{bmatrix} x_1 & x_0 \end{bmatrix} \begin{bmatrix} a & 1 \\ b & 0 \end{bmatrix}^{i-1}$$

- Parallel prefix with **multiplication operator** can then be applied.

Parallel Search

Problem description

- Given a sorted list A of size n and an element x
- find the index k , such that

$$a_k \leq x < a_{k+1}$$

Best sequential solution:

- Use binary search
- $O(\log_2(n))$ time

Parallel search: How do we design a parallel algorithm for the search problem?

That's all for today!